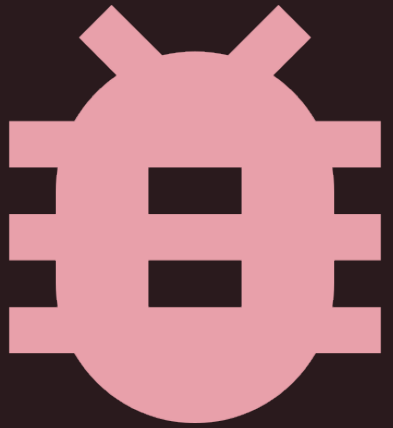
LAB - DAY 6

Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

Find and fix injection and scripting flaws

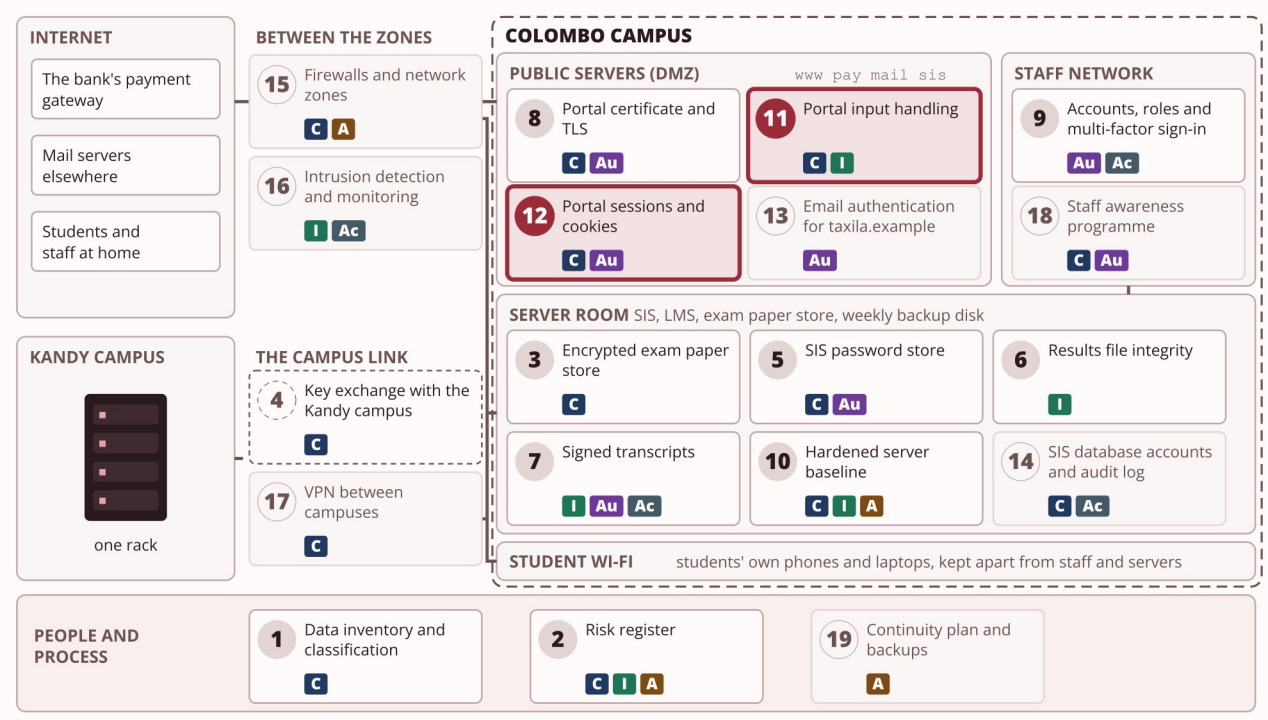


Adds to the register: **11** **12**

Floor tier: every step | **Full tier: recommended**

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

The security register



TODAY

11 Portal input handling

12 Portal sessions and cookies

KEY

- Added today
- Added earlier
- Concept only
- Not yet added

IT

HEAD OF IT

Find me the line that trusts what a student types, and show me it fixed.

The Head of IT, at the start of the lab.

Before you start

```
# both tiers: your index number seeds your own practice record
# floor tier: open the sqli-login and output-encoding demos
# full tier: snapshot day-6-start, then download taxila-portal.py
```

You never craft an attack and you never type one. A prepared test probes; you read the report, find the line, and fix it.

SEGMENT 1 - 40 MIN

Read the login code and find the injection flaw

Both tiers

Run the test and find the injection flaw

1 Run the provided test with your index; read its report.

Expected: The probe returns every record in the table; a normal sign-in returns 1.

Floor path: Run the login test in `sqli-login`.

2 Open the code; find the line that joins input into the query.

Expected: The marked flawed line inside `flawed_login`.

Floor path: Step the test value through the joined query in the demo.

```
$ python3 taxila-portal.py --test --index COHNDNE999X000
the prepared login probe returned 6 of 6 records
RESULT: OPEN. The login is injectable.
```

The test value is prepared and never shown as a ready string. The report shows only the record counts.

SEGMENT 2 - 45 MIN

Fix the query and prove it with the test

Both tiers

Apply the parameterised query and prove it

3 Set `SAFE_LOGIN = True`; save; run the test again.

Expected: The probe returns 0 records; a normal sign-in still returns 1.

Floor path: Choose the parameterised query in the demo and run the test.

```
$ python3 taxila-portal.py --test --index COHNDNE999X000
the prepared login probe returned 0 of 6 records
RESULT: CLOSED. A normal sign-in still works.
```

SEGMENT 3 - 40 MIN

Find and fix the echo flaw

Both tiers

Find and fix the echo flaw

4 Run the note test; find the echo line; set `SAFE_ECHO = True`.

Expected: Before: the bold note reaches the page as markup. After: shown as plain text, with your index.

Floor path: Toggle encoding in `output-encoding`.

```
Note page: the bold note reached the page as markup (OPEN)
after output encoding: shown as plain text (CLOSED)
```

The note is bold text only. A script in a note would run in the same place; encoding shows it as text instead.

SEGMENT 4 - 25 MIN

Prove it, break it, reset and write up

Both tiers

Prove it, then break it

PROVE IT

- Login test: every record before the fix, including your seeded record
- Login test: 0 records after, 1 for a normal sign-in
- Note test: page changed before encoding, plain text after

× BREAK IT

- Set `SAFE_LOGIN` back to `False`; run the test
- The probe returns every record again
- Set it to `True`; the leak is closed

Reset, and the rules

RESET

- Keep your two test reports and your register lines first
- Full tier: restore `day-6-start`
- Floor tier: reload the two demos

RULES OF ENGAGEMENT

- No tool in this lab can affect anyone else's system
- A prepared test probes; you read, find and fix

Record your evidence

EVIDENCE

- Your seeded record and both login counts (your personal count before, 0 after)
- The test report before and after your query fix, showing your seeded record leaking and then not
- The flawed line and your fixed line
- The echo test before and after output encoding
- The hand-drawn code-and-data sketch
- Your 120-word explanation

Questions & Discussion

Next: Day 7, email and databases. You see why an email cannot prove who sent it.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision 9f64626